

CSC 648 / CSC 848

Section 03

Milestone 2 — Version 1 (M2v1)

Project: **Home4U**

AI-Assisted Interior Design Recommendation Platform

Team Number: **4**

Team Alias: **Vibecoding for Internship**

Team Members and Roles

Caleb Ponce — Team Lead, Backend/Auth, Deployment

Mason Lee — Frontend/UI

Christopher Quach — Backend/API & Database Alignment

Tyler Morris — QA / Smoke Tests / Deployment Verification

Dias Almat — Data (Styles/Tags/Search) & Recommendations

Team Lead Email

cponce8@sfsu.edu

Date

March 4, 2026

Revision History

Version	Date	Author(s)	Description
M2v1	Mar 4, 2026	Caleb Ponce	Final Milestone 2 submission
M2v0.5	Mar 1, 2026		
M2v0.9	Mar 3, 2026		
M1v2	Feb 19, 2026		
M2v0.7	Mar 2, 2026		
M1v1	Feb 8, 2026		

M1v0.1 Feb 1, 2026

document structure Team Revised Milestone

Team Completed architecture diagrams, APIs, and deployment documentation 1 based on instructor feedback Team Initial

Team Drafted data definitions and functional Milestone 1 submission

requirements Team Created Milestone 2

Team Initial project draft

Table of Contents

CSC648/CSC8481 Section 031 Milestone 2—Version 1 (M2v1) 1 Team Members and Roles 1

Team Lead Email 1 Date 1 Revision History 1 Table of Contents 2 1. Data Definitions 3

RoomProject 4 Style 4 Tag 4 StyleTag 5 RoomTag 5 ResemblanceScore 5 Recommendation 6

ProductItem 6 **2. Prioritized Functional Requirements 6** Priority 1—Critical 6 Priority 2—

Important 7 Priority 3—Nice to Have 7 **3. Mockups and Storyboards 8** PrimaryUserWorkflow 8

LoginPage 8 Dashboard 9 CreateRoomProject 9 RecommendationsPage 10

4. High-Level System Design 10 4.1 System Architecture Overview 10

4.2 Backend Technologies 11 4.3 Network and Deployment Design 12

4.4 High-Level APIs and Algorithms 13 4.5 Database Architecture 14

EntityRelationshipDiagram 14

5. Key Project Risks 15 6. Project Management 16 7. Team Contributions 16

Caleb Ponce — Team Lead, Backend/Auth, Deployment 17 Mason Lee — Frontend/UI

Development 17 Christopher Quach — Backend API and Database Integration 17 Tyler

Morris — Quality Assurance and Deployment Verification 18 Dias Almat — Data

Organization and Recommendation Logic 18

1. Data Definitions

The following entities represent the core database schema used by the Home4U backend.

User

Represents a registered user of the platform.

Attributes

id

email
password_hash
created_at

Relationship

User → one-to-many → RoomProject

RoomProject

Represents a user-created room design project.

Attributes

id
user_id (FK → User)
room_type
budget
photo_url
created_at

Relationship

RoomProject → RoomTag
RoomProject → Recommendation
RoomProject → ResemblanceScore

Style

Represents an interior design style available for

recommendation. Attributes

id
name
description

Tag

Represents descriptive keywords used to categorize styles.

Attributes

id
name

StyleTag

Associates tags with styles.

Attributes

id
style_id
tag_id
weight

RoomTag

Associates tags with a room project.

Attributes

id
room_project_id
tag_id
is_confirmed

ResemblanceScore

Stores similarity scores between room projects and

styles. Attributes

id
room_project_id
style_id
score_value
created_at

Recommendation

Stores design recommendations generated for a project.

Attributes

id
room_project_id
description
priority_score
estimated_cost
is_completed
created_at

ProductItem

Represents product suggestions associated with a style.

Attributes

id
style_id
name
estimated_cost
url
Image_url

2. Prioritized Functional Requirements

Priority 1 — Critical

FR1 — User Signup

Users shall be able to register using an email and password.

FR2 — User Login

Users shall authenticate and receive a JWT token.

FR3 — Create Room Project

Authenticated users shall create room design projects.

FR4 — List Room Projects

Users shall retrieve projects associated with their account.

FR5 — Retrieve Styles

Users shall retrieve available interior design styles.

FR6 — Retrieve Style Tags

Users shall retrieve tags associated with styles.

FR7 — Search Styles

Users shall search styles using text queries.

FR8 — Generate Recommendations

Users shall receive recommendations associated with a project.

FR9 — Enforce Project Ownership (NEW)

The system shall allow authenticated users to view, update, or delete only the room projects that belong to their account.

FR10 — Validate User Credentials (NEW)

The system shall require users to provide a valid authentication token to access protected resources.

FR11 — Persist Room Project Data (NEW)

The system shall save room project information when a user creates or updates a project and allow the user to retrieve the saved project in future requests.

FR12 — Associate Recommendations with Styles (NEW)

The system shall include associated style information with each generated recommendation returned to the user.

Priority 2 — Important

FR13 — Update Room Project

Users shall modify project information.

FR14 — Delete Room Project

Users shall delete projects.

FR15 — Upload Room Photo

Users shall upload a room image which is stored locally.

FR16 — Assign Tags to Room Project

Users shall associate tags with projects.

FR17 — Compute Resemblance Scores

The system shall calculate similarity scores between room tags and style tags.

FR18 — Mark Recommendation Complete

Users shall mark recommendations as completed.

FR19 — Update Recommendation Status

Users shall update the status of a recommendation.

FR20 — Retrieve Tags

Users shall retrieve the set of available tags.

FR21 — Search Tags

Users shall search tags using text queries.

FR22 — Associate Style Tags with Styles

The system shall associate tags with styles.

FR23 — Support Multiple Room Photos per Project

The system shall allow a room project to have multiple uploaded room photos.

Priority 3 — Nice to Have

FR24 — Product Suggestions

The system may recommend purchasable furniture products.

FR25 — AI Tag Suggestions

Future versions may include automated tag generation using computer vision.

3. Mockups and Storyboards

This section presents simplified wireframe layouts representing the major pages of the Home4U platform.

Primary User Workflow

Login



Dashboard



Create Room Project



Upload Room Photo



Search/Browse Styles



View Recommendations

Login Page

Home4U

Email

Password

[Login]

Don't have an account?

[Sign Up]

Dashboard

Dashboard

+ Create Room Project

Living Room
Budget: \$2000

Bedroom
Budget: \$1500

Office
Budget: \$1200

[Logout]

Create Room Project

Create Room Project

Room Type
[Living Room ▼]

Budget
[\$2000]

Upload Room Photo
[Choose File]

[Create]

Recommendations Page

Design Recommendations

Room: Living Room

1. Replace curtains with linen
Estimated Cost: \$120
2. Add pendant lighting
Estimated Cost: \$180

3. Replace rug with neutral style
Estimated Cost: \$150

4. High-Level System Design

4.1 System Architecture Overview

Home4U follows a **client-server architecture** in which the frontend application communicates with the backend through REST APIs. The architecture separates the user interface, application logic, and data storage layers to improve scalability and maintainability.

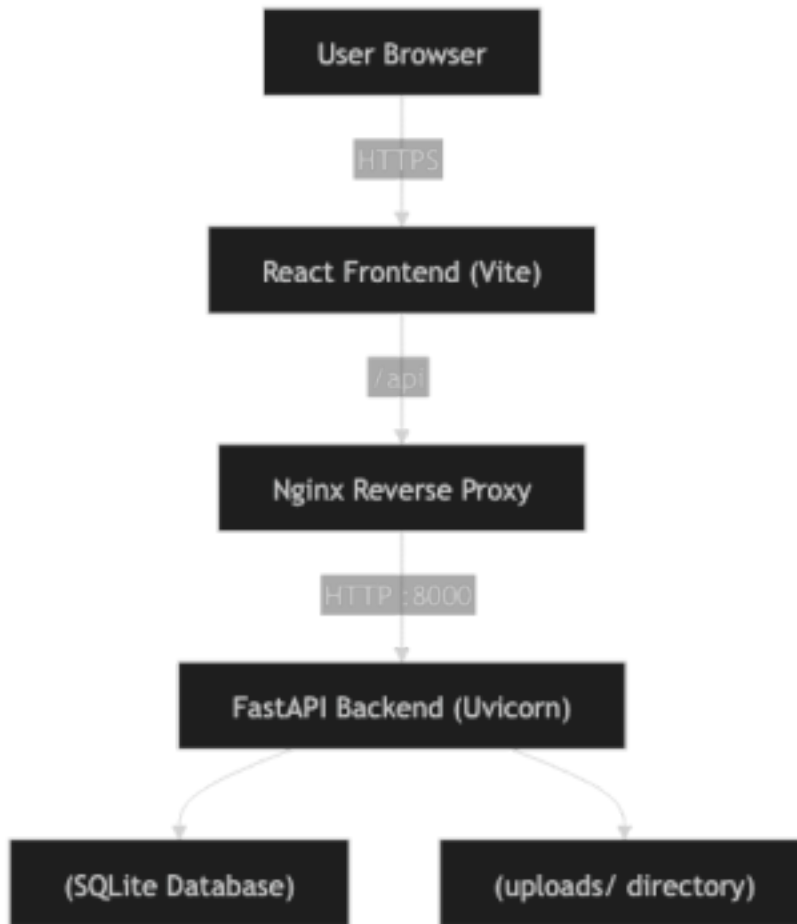
The frontend application is implemented using **React with the Vite build system**, which communicates with the backend through HTTP requests. These requests are routed through **Nginx**, which acts as a reverse proxy and handles routing and request forwarding to the FastAPI backend.

The backend service is implemented using **FastAPI**, a high-performance Python web framework. The backend processes requests, performs authentication using JWT tokens, executes search and recommendation algorithms, and communicates with the database.

Data persistence for the vertical prototype is handled using **SQLite**, which stores user data, projects, styles, tags, and recommendations. Uploaded room images are stored in a local **uploads directory** referenced by the database.

Future versions of the system will migrate the database to **PostgreSQL** and store media files in **AWS S3** to improve scalability and reliability.

Figure 1 — Home4U System Architecture



4.2 Backend Technologies

The backend of the Home4U system is implemented using modern web technologies designed for scalability and maintainability.

Backend Framework

FastAPI (Python) is used to implement the REST API services. FastAPI provides automatic request validation, asynchronous support, and high performance.

Authentication

User authentication is implemented using:

- bcrypt password hashing
- JWT authentication tokens (HS256)

- case-insensitive email normalization

These mechanisms ensure that user credentials are securely stored and verified.

Database

For the vertical prototype, the system uses **SQLite**, which allows rapid development and simplified deployment. SQLite stores all core entities including users, projects, styles, tags, and recommendations.

Future production versions will migrate to **PostgreSQL**, which provides stronger concurrency support and improved scalability.

Media Storage

Room images uploaded by users are stored locally in an **uploads directory** on the server. The database stores a reference to the image location using the `photo_url` field.

4.3 Network and Deployment Design

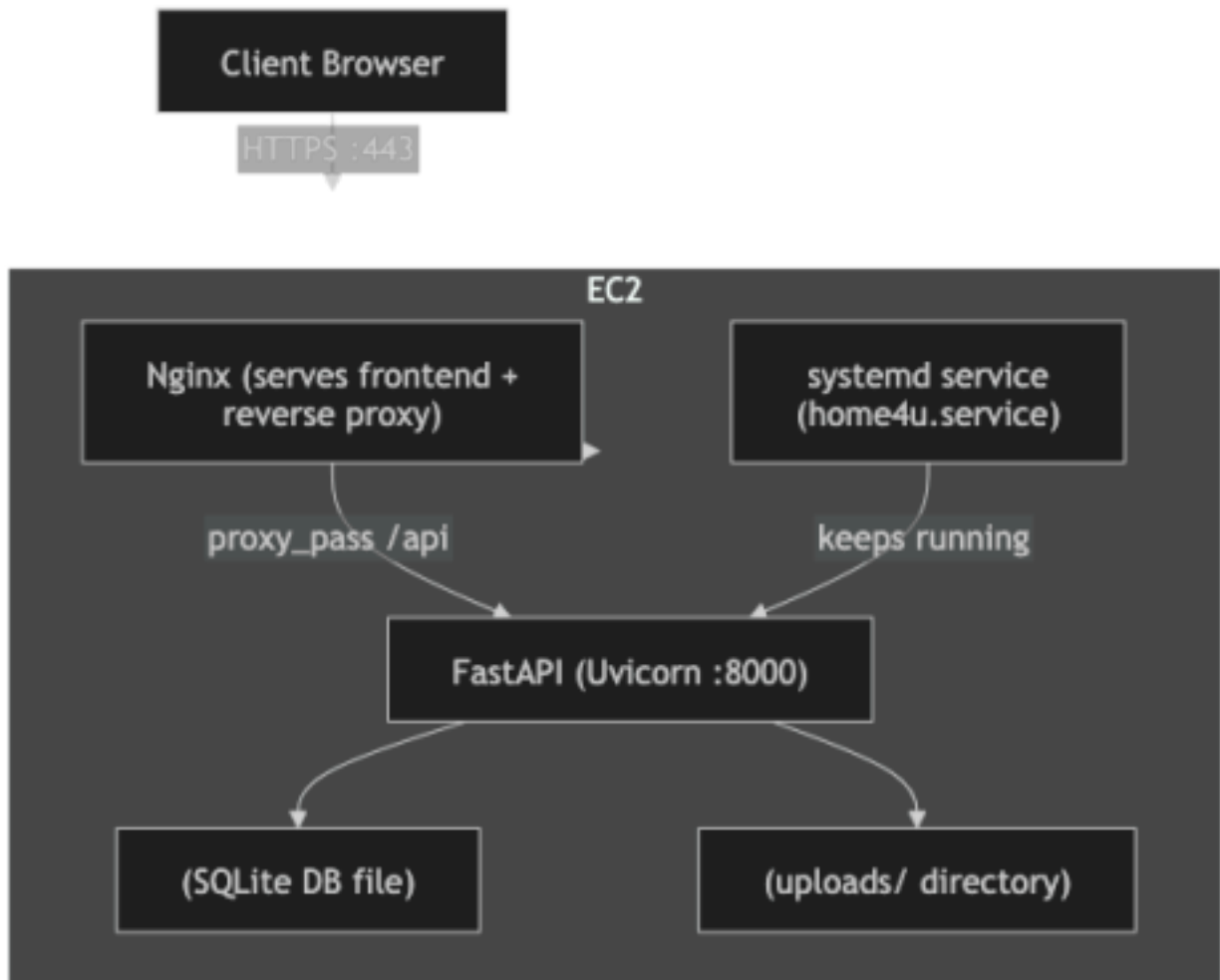
The Home4U platform is deployed on an **AWS EC2 instance running Ubuntu**. The deployment architecture uses a reverse proxy configuration to separate web traffic from application logic.

Incoming HTTP or HTTPS requests are received by **Nginx**, which serves the frontend application and forwards API requests to the FastAPI backend running on port 8000.

The backend service is managed by a **systemd service**, which ensures that the FastAPI server automatically restarts if the process fails or if the system reboots.

This architecture provides a simple but reliable infrastructure suitable for the vertical prototype while allowing future scaling.

Figure 2 — Home4U Deployment Architecture



4.4 High-Level APIs and Algorithms

The system uses the OpenAI Vision API to analyze a user's uploaded room images, using the api it then analyzes the user current room and generates descriptive tags representing the rooms visual features such as furniture types, colors, layout, and lighting. These tags are reviewed and validated before being used as structured inputs for style comparison and recommendation generation.

The weighted resemblance scoring algorithm is used to evaluate the similarity between a user's room and selected interior styles. The algorithm compares validated room tags against predefined style characteristics for each specific style such as modern, industrial, etc. Each matching feature contributes to a weighted score based on its visual importance, producing an overall similarity value that reflects how closely the room matches each selected style.

This similarity score is used to determine which styles most closely align with the current room configuration and to guide the generation of targeted improvement recommendations.

The system uses a budget-aware prioritization algorithm to generate recommendations that remain within a user-defined spending limit. Based on the calculated similarity scores, the

system identifies potential room improvements that increase resemblance to the selected style. Each recommendation has an estimated cost to ensure that we do not go over the predefined budget if applicable.

4.5 Database Architecture

The Home4U database schema is designed using a normalized relational structure. The schema includes entities for users, room projects, styles, tags, recommendations, and product items.

Join tables such as **RoomTag** and **StyleTag** are used to represent many-to-many relationships between projects, styles, and tags.

Similarity between room projects and styles is represented using the **ResemblanceScore** entity, which stores computed similarity scores used by the recommendation system.

Entity Relationship Diagram

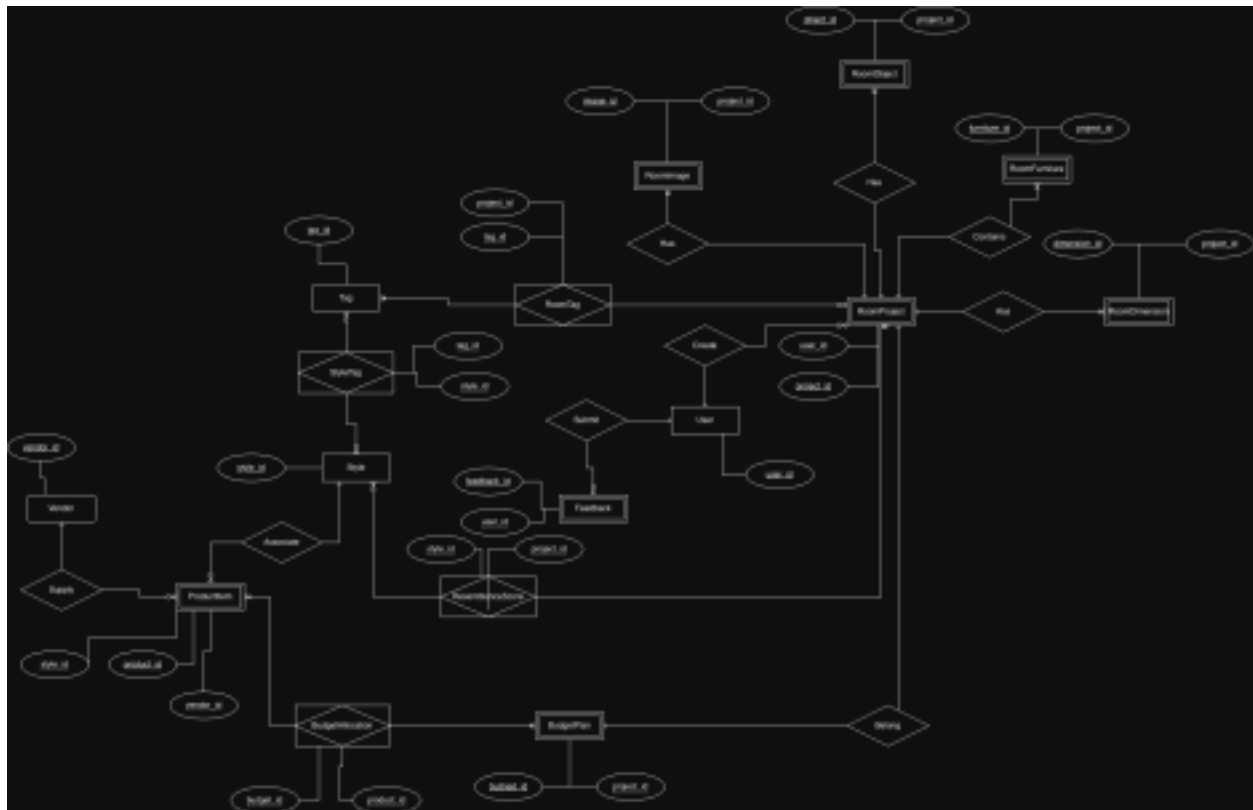


Figure 3 — Home4U Database Entity Relationship Diagram

5. Key Project Risks

Several potential risks could impact the development and deployment of the Home4U system.

Data Consistency Risks

Differences between frontend data handling and backend database schemas could result in inconsistent data states. This risk is mitigated by enforcing consistent API contracts and validating data using backend schemas.

Authentication Security Risks

Improper handling of authentication tokens could expose the system to security vulnerabilities. To mitigate this risk, the system uses bcrypt password hashing and JWT tokens with expiration times.

Single-Instance Infrastructure Risk

The vertical prototype currently runs on a single EC2 instance using SQLite. This creates a potential risk of downtime or data loss. Future versions will migrate to PostgreSQL and distributed cloud infrastructure.

Media Storage Growth

As more users upload room images, storage requirements will increase. Future versions of the system will migrate media storage to scalable cloud storage services such as AWS S3.

Team Coordination Risk

As multiple team members contribute to the same codebase, integration issues may arise. This risk is mitigated by using GitHub version control, pull requests, and regular team meetings.

6. Project Management

Development Tools

GitHub — version control

Notion — project tracking

Discord — team communication

Development Process

Trunk-based development with small pull requests and code reviews.

Definition of Done

Feature merged to repository

Deployment successful

Smoke tests pass

Core workflow operational

7. Team Contributions

Caleb Ponce — Team Lead, Backend/Auth, Deployment

Caleb Ponce served as the Team Lead and was responsible for coordinating team tasks and ensuring the successful completion of Milestone 2 documentation.

Technical contributions included implementing the authentication system using bcrypt password hashing and JWT tokens, configuring the FastAPI backend structure, and integrating authentication endpoints such as /auth/signup, /auth/login, and /auth/me. Caleb also configured the deployment environment on an AWS EC2 instance, including setting up the Nginx reverse proxy and creating a systemd service to ensure the FastAPI backend automatically restarts after server reboots.

In addition, Caleb integrated backend API routes with the frontend application, verified database connectivity with SQLite, and prepared the system architecture, deployment architecture, and ERD diagrams included in this milestone documentation.

Mason Lee — Frontend/UI Development

Mason Lee implemented the primary frontend interface for the Home4U platform using React with the Vite build system.

Responsibilities included creating the login and signup interface components, building the dashboard interface for listing room projects, and implementing the form interface for creating new room projects. Mason also integrated the frontend application with backend API endpoints, ensuring that authentication requests and project creation requests were properly sent and handled.

Additional work included implementing user interface navigation between pages and structuring the layout of the dashboard and recommendation display views.

Christopher Quach — Backend API and Database Integration

Christopher Quach focused on backend API development and database integration.

He implemented several core API routes responsible for project management and style retrieval, including the /projects and /styles endpoints. Christopher also worked on connecting the FastAPI backend to the SQLite database, ensuring that entities such as RoomProject, Style, and Tag were correctly stored and retrieved.

He assisted in aligning the database schema with the application data models and helped verify that relationships between entities such as RoomTag, StyleTag, and ResemblanceScore were correctly represented in the backend.

Tyler Morris — Quality Assurance and Deployment Verification

Tyler Morris was responsible for testing and validating the system functionality during development.

His contributions included performing API testing using curl commands to verify that authentication and project management endpoints functioned correctly. Tyler also assisted in verifying deployment functionality on the EC2 server by testing that the Nginx reverse proxy properly routed requests to the FastAPI backend.

In addition, Tyler helped confirm that the systemd service correctly restarted the backend service after server restarts and performed smoke tests to ensure the core workflow of login, project creation, and recommendation retrieval functioned correctly.

Dias Almat — Data Organization and Entity Relationship Diagram

Dias Almat contributed to organizing the style and tag data used by the recommendation system.

Responsibilities included defining the relationships between styles and tags, assisting in structuring the StyleTag and RoomTag entities, and helping define how tag relationships contribute to similarity scoring between room projects and interior design styles.

Dias also assisted with refining the recommendation logic by helping define how resemblance scores are calculated and how recommendations are prioritized based on style similarity and estimated cost.